Outcome-First Agent Loop

How the controller is built: architecture, code map and features

Pardeep Singh · sandhupardeep300@gmail.com · 8 November 2026

The controller is an always-on agent that steers outbound-calling workflows toward better outcomes. On every tick it reads new call events, detects problems (a spam-flagged caller ID, a hallucinating prompt), runs experiments on what customers hear, asks a human for permission in a form they can say yes to, applies what was approved, and rolls back what stops working. It is about 4,000 lines of Python 3.12 (standard library plus `httpx`), keeps all of its state in one SQLite file, and talks to the simulator only over HTTP. Every decision can be explained afterwards from the store alone.

Results on the public scenarios, graded by the simulator against the no-op baseline:

Scenario	Leads concluded vs baseline	Changes applied	Harmful changes
public_01 (3 workflows, 8 days)	**+16.1%**	12	1 small (−0.022: an undecided test ended on the incumbent)
public_02 (incidents, 10 days)	**+28.4%**	14	0
public_03_load (12 workflows)	**+21.8%**	57	1 (a 10% probe of an untested caller ID, removed)
public_02 with 8 hard kills	**+27.4%**	14	1; all 7 crash invariants held

1. Architecture

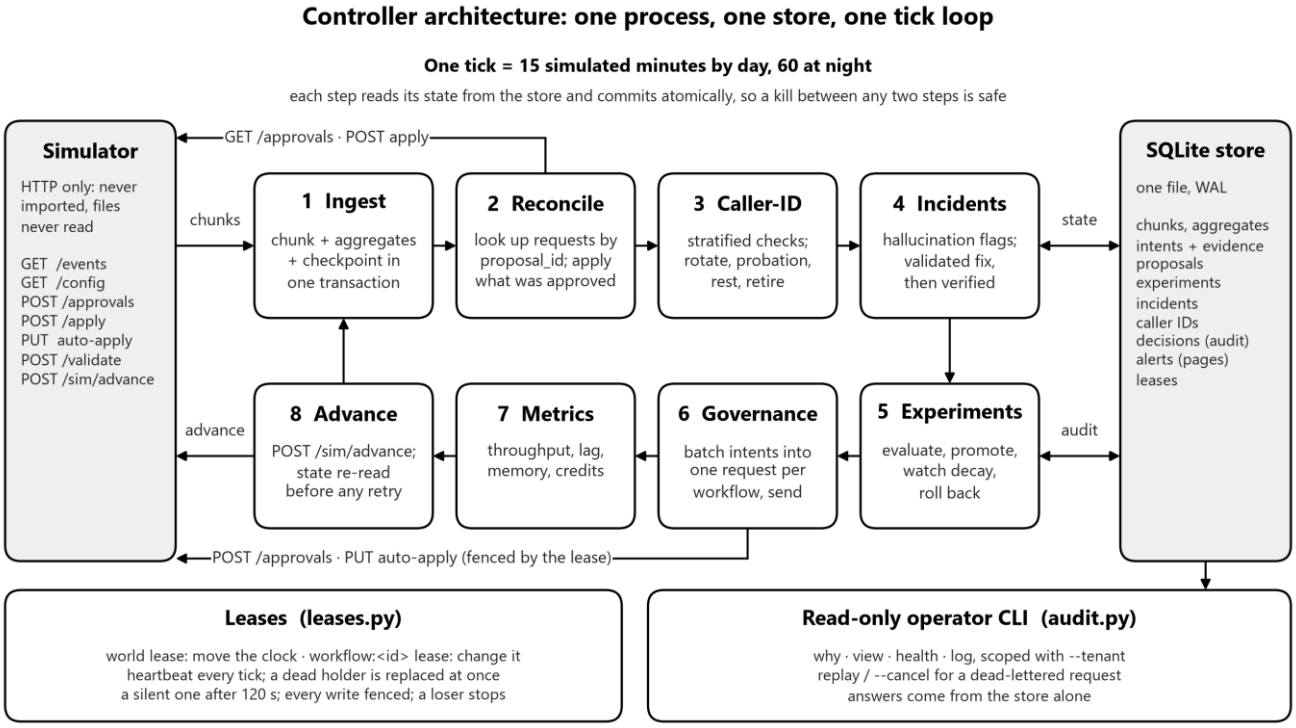


Figure 1. One process, one store, one tick loop. The controller advances the simulator's clock itself, so it never falls behind the events it acts on.

Three rules shape the whole codebase:

- **No in-memory truth.** Every step reads its state from the SQLite store and commits atomically. Killing the process between any two steps loses at most that step's work, which is redone on restart.
- **Only governance writes a workflow's config.** Detectors and experiments file **intents** (replayable ops plus evidence); governance batches them into one request per workflow and picks the path.
- **HTTP only.** The controller never imports the simulator, reads its files or looks at scenario ground truth. Everything it knows comes from events and API responses.

2. The tick loop

One tick is 15 simulated minutes by day and 60 at night. The steps always run in this order:

1. **Ingest.** Fetch new event chunks; each chunk's aggregates and checkpoint commit in one transaction, so every event counts exactly once.
2. **Reconcile.** Look every in-flight request up by `proposal_id`, apply approved ones inside their 2-hour window, and adopt requests whose response was lost.

3. **Caller-ID health.** Compare each number with the others in the same cohort and hour; rotate, rest or retire numbers that go bad.
4. **Incidents.** Detect hallucination spikes with an exact binomial test, validate a fix before asking for it, and verify it afterwards.
5. **Experiments.** Evaluate welcome-message and voice tests with always-valid statistics; promote, watch for decay, roll back.
6. **Governance.** Turn ready intents into batched requests on the right path, at a time the approver can act on them.
7. **Metrics, then advance.** Record throughput, lag, memory and credits; then move the clock. `/sim/state` is re-read before any retry, so time never moves twice.

3. Code map

Module (controller/)	Lines	Responsibility
`loop.py`	263	The tick loop, the store-to-world continuity check, loop metrics
`governance.py`	577	Intents to batched requests, path choice, timing and quota, reconciliation, apply, retry backoff, dead-lettering
`experiments.py`	900	Experiment lifecycle, segment cohorts, promotion, holdout, decay, rollback
`incidents.py`	487	Hallucination response, validation and credit accounting, fix verification, drift alerts
`callerid.py`	370	Caller-ID state machine, stratified detection, rotation in `max_shift` steps
`stats.py`	156	mSPRT confidence sequences, Mantel–Haenszel, exact binomial tails, Dunnett allocation
`ingest.py`	173	Chunk ingestion, lead and feature joins, hourly and segment aggregates
`leases.py`	119	One process per world and per workflow: heartbeats, takeover, fencing before writes
`db.py`	165	Schema, in-place migrations, nested atomic transactions
`api.py`	115	HTTP client with per-call retry rules (advance never twice, paid calls never retried)
`estimates.py`	107	Affected leads, dry-run replay and impact score attached to every request
`audit.py`	203	Read-only `why`, `view`, `health` and `log`, scoped per client with `--tenant`
`settings.py`	124	Every threshold with the reason for it; tenant rules (Northwind, Kestrel)

4. From a signal to a live change

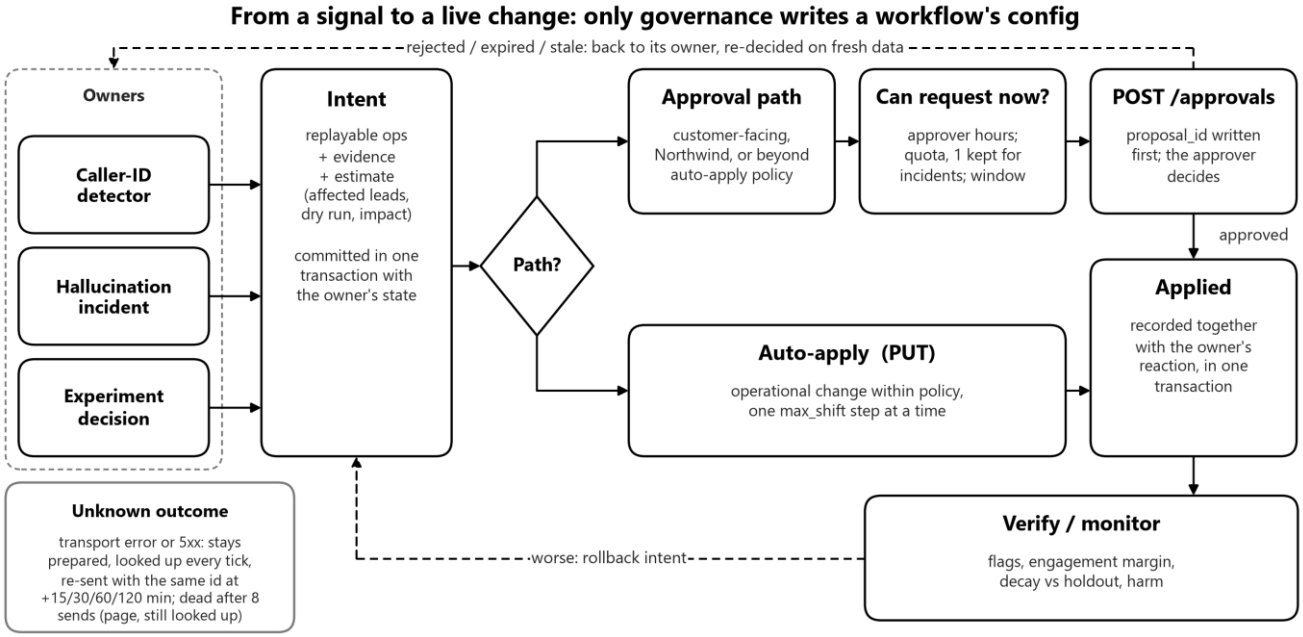


Figure 2. Every change goes through governance; every outcome flows back to the owner that asked for it.

An intent commits in the same transaction as the state of the owner that filed it, so a crash leaves neither behind. Governance then picks the path. Anything customers hear (welcome message, voice, prompt, cohorts) goes to a human, and Northwind requires a human even for rollbacks. Operational changes within policy, such as schedule and caller-ID shares, auto-apply one `max_shift` step at a time. A request is sent only when the approver can act on it: inside approver hours, under the daily quota with one slot kept for incidents, and with enough of the window left. Each request carries its diff, affected leads, a dry run, an impact score and the evidence. Its `proposal_id` is written down before it is sent, so a lost response is looked up rather than sent twice.

5. Features

- **Experiments with honest statistics.** Arms are compared only within (config version, day) strata and combined with Mantel–Haenszel, so a traffic-share change during a trend cannot fake a lift. A winner needs an always-valid mSPRT confidence sequence that excludes zero (Bonferroni over challengers), at least 300 trials per arm, a guardrail KPI shown non-inferior, and a lift that still holds over the last 24 hours. Traffic is split by the square-root rule; segment winners get their own cohort; promotion keeps a 10% holdout, and a drift-adjusted CUSUM against it triggers rollback on decay.
- **Caller-ID health.** Each number moves from untested to probation (10% of traffic) to healthy, or to resting or out. Detection is stratified by cohort and hour ($z \le -4.5$, about 3×10^{-6} false alarms per look). In public_02 the spam-flagged number was out of rotation 2 h 30 min after the flag.
- **Hallucination incident response.** An exact binomial test on the flag rate opens the incident. Known-good prompts from production are tried first and LLM suggestions only if none exists. Every candidate is validated (n = 400) inside a 25-credit ceiling with a 30-credit reserve, charged before each call. A fix closes only after it is verified on 300+ calls against the workflow's engagement margin. MTTR in public_02: 150 minutes.
- **Crash safety.** Idempotency keys are written before sending; an outcome and the owner's reaction commit together; paid calls are settled against the simulator's balance; a store is resumed only if it matches the simulator's own records (chunks, config history, approvals), otherwise it is archived.
- **One owner per workflow.** A `world` lease to move the clock and a `workflow:<id>` lease to change a workflow, renewed every tick. A dead holder is replaced at once and a silent one after 120 s. Every write is fenced on the lease, and a process that loses one stops.
- **Retries without duplicates.** An ambiguous failure (transport error, 5xx) is never treated as a failure. The request is looked up every tick and re-sent with the same id at 15/30/60/120-minute gaps. After 8 sends it is dead-lettered (a human is paged; it is still looked up). `replay` re-sends it, and `--cancel` hands the work back once a lookup confirms the request never landed.
- **Humans stay in charge.** Supervisor edits are never overwritten, prompt changes always carry a validation run, and `why` reconstructs any decision at any minute from the store. `--tenant` limits a client to its own workflows, decisions and alerts.

6. Verification and running it

- **83 tests** (77 for the controller, 6 for the simulator), including an end-to-end smoke run of the wired controller and every failure path above: lost responses, crashes between writes, competing processes, statistical edge cases.
- **Crash test:** 8 hard kills during public_02; no change applied twice, every approval reconciled, nothing lapsed, every chunk ingested once.
- **Load:** about 79,000 events/s, 19–23 s of wall clock per simulated day including the simulator (target under 600 s), 759 MB peak memory.
- **Evidence harness:** refuses to grade a run unless the controller exited 0 and the scenario finished, and never overwrites earlier evidence without `--overwrite`.

```
pip install -r requirements.txt
python scripts/run_scenario.py scenarios/public_02_incidents.yaml --tag my_run   # serve, run, grade
python scripts/crash_test.py scenarios/public_02_incidents.yaml                  # 8 kills, 7 checks
python -m pytest -q tests
python -m controller why --db state/my_run.db --workflow ks_cancel --at "D3 20:00"
```

7. Known limits and next steps

- **Partial decay rolls back slowly.** Decay is judged against a shrunk lift, so a winner that keeps part of its lift stays live (in public_01 a fading variant stayed at 90% from day 4 to day 6). A 20% holdout or a scheduled re-test would sharpen this.
- **Guardrail evidence is expensive.** Showing a pickup drop under 2 pp with an always-valid bound needs about 6–7k dials per arm, so promotions wait days. A variance-reduced estimator would shorten the wait.
- **Statistics are asymptotic.** The sequences use a normal approximation; decisions are gated at 300+ trials per arm and the error rates were checked by simulation. Segment claims are single-feature with Bonferroni.
- **Path to production.** Move the store to Postgres (workflows claimed with `SELECT … FOR UPDATE SKIP LOCKED`), replace the chunk checkpoint with an event-level dedupe ledger, shadow-run for a week in propose-only mode, and switch on caller-ID rotation first, since it was the largest win.